

# Python PBL 01

« Программа учёта оценок студентов на основе ООП  
в Python »



SK Shieldus

Rookies, 30-й набор (личный номер 14)

**Ким Джэ-Ук**

# Решение PBL 01

```
# стандартная библиотека для работы с операционной системой,
# например с путями к файлам
import os
# библиотека для переноса длинного текста по заданной ширине
import textwrap
# класс для хранения и анализа оценок студентов
class StudentScores:
    # конструктор, который выполняется при создании объекта
    def __init__(self, filename="scores_Korean.txt"):
        # сохраняем имя файла с оценками как атрибут объекта
        self.filename = filename
        # пустой словарь, ключ это имя студента, значение это балл
        self.scores = {}
        # вызываем внутренний метод, который читает данные из файла
        self._load()
    # внутренний метод только для загрузки данных из файла
    def _load(self):
        # при открытии файла может возникнуть ошибка,
        # поэтому начинаем обработку исключений
        try:
            # открываем файл с оценками в режиме чтения
            with open(self.filename, "r", encoding="utf-8") as f:
                # читаем файл построчно по порядку
                for line in f:
                    # убираем пробелы по краям и знак перевода строки
                    line = line.strip()
                    # если строка пустая
                    if not line:
                        # ничего не делаем и идём к следующей строке
                        continue
                    # при делении строки на имя и балл возможна ошибка,
                    # поэтому тоже обрабатываем исключения
                    try:
                        # делим строку на имя и балл по запятой
                        name, score = line.split(",", 1)
                        # убираем пробелы по краям имени
                        name = name.strip()
                        # превращаем балл из строки в вещественное число
                        score = float(score.strip())
                        # только если имя не пустое
                        if name:
                            # сохраняем имя и балл в словарь
                            self.scores[name] = score
                    except:
                        # если формат неверный и преобразование не удалось
```

```

        except ValueError:
            # пропускаем эту строку и идём к следующей
            continue

# если файла нет или его невозможно открыть
except (FileNotFoundError, OSError):
    # данных нет, поэтому оставляем пустой словарь
    self.scores = {}

# метод, который вычисляет средний балл студентов
def calculate_average(self):
    # если нет ни одной записи об оценках
    if not self.scores:
        # возвращаем средний балл 0.0
        return 0.0

    # делим сумму всех баллов на число студентов
    return sum(self.scores.values()) / len(self.scores)

# метод, который находит студентов с баллом не ниже среднего
def get_above_average(self):
    # вычисляем общий средний балл и сохраняем его в avg
    avg = self.calculate_average()
    # возвращаем список имен студентов не ниже среднего
    return [name for name, score in self.scores.items() if score >= avg]

# метод, который сохраняет в файл студентов ниже среднего
def save_below_average(self, output_filename="below_average_Korean.txt"):
    # вычисляем средний балл
    avg = self.calculate_average()
    # список пар из имени и балла для студентов ниже среднего
    below = [(name, score) for name, score in self.scores.items()
              if score < avg]

    # если таких студентов нет ни одного
    if not below:
        # ничего не делаем и выходим из метода
        return

    # открываем выходной файл в режиме записи
    with open(output_filename, "w", encoding="utf-8") as f:
        # перебираем список студентов с баллом ниже среднего
        for name, score in below:
            # пишем имя и балл через запятую одной строкой
            f.write(f"{name},{score}\n")

# метод, который выводит на экран результаты анализа
def print_summary(self):
    # если данных об оценках нет
    if not self.scores:
        # выводим поясняющее сообщение
        print("Нет данных для анализа.")
        # выходим из метода
        return

```

```

# вычисляем средний балл
avg = self.calculate_average()
# получаем список студентов с баллом не ниже среднего
above = self.get_above_average()
# соединяем имена и переносим их по ширине 70 знаков,
# чтобы длинная строка не выходила за край страницы
names = textwrap.fill(", ".join(above), width=70) if above else "нет"
# выводим заголовок результатов анализа
print("Результаты анализа успеваемости студентов")
# выводим общее число студентов
print(f"Общее число студентов {len(self.scores)} чел.")
# выводим средний балл с двумя знаками после запятой
print(f"Средний балл {avg:.2f}")
# выводим количество студентов не ниже среднего
print(f"Студентов не ниже среднего {len(above)} чел.")
# выводим их имена
print(names)

# условие, при котором код ниже выполняется
# только при прямом запуске этого файла
if __name__ == "__main__":
    input_file = "scores_Korean.txt"
    output_file = "below_average_Korean.txt"
    # в среде Quarto, в отличие от файла .py в VSCode,
    # переменная __file__ часто оказывается не определена
    # то есть если строить путь от расположения самого файла Python,
    # как показано ниже, возникает ошибка NameError
    # поэтому имя файла используется напрямую относительно
    # рабочей папки, то есть текущего места запуска

    # base_dir = os.path.dirname(os.path.abspath(__file__))
    # получаем путь к папке, где находится этот файл Python
    # input_file = os.path.join(base_dir, "scores_Korean.txt")
    # строим путь к входному файлу от текущей папки
    # output_file = os.path.join(base_dir, "below_average_Korean.txt")
    # строим путь к выходному файлу от текущей папки
    # создаём объект StudentScores и передаём ему входной файл
    s = StudentScores(input_file)
    # выводим на экран сводку с результатами анализа
    s.print_summary()
    # сохраняем в файл студентов с баллом ниже среднего
    s.save_below_average(output_file)

```

Результаты анализа успеваемости студентов  
 Общее число студентов 25 чел.  
 Средний балл 84.76  
 Студентов не ниже среднего 13 чел.

윤서연, 한지우, 임수빈, 배가은, 조예린, 장하린, 고유진, 류나영, 김하윤, 박지민, 최유나, 백서진, 노아린

**По ходу вычислений и по результатам выполнения кода на Python можно убедиться, что все цели, поставленные в задании, полностью достигнуты.**

## Подробное содержание файла scores\_korean.txt

```
# имя текстового файла, он должен лежать в текущей рабочей папке
path = "scores_korean.txt"
# открываем файл в режиме чтения и указываем кодировку utf-8,
# чтобы текст не искажался
with open(path, "r", encoding="utf-8") as f:
    # читаем всё содержимое файла сразу как одну строку
    content = f.read()
print(content)
```

홍지훈,81  
윤서연,93  
정민재,76  
한지우,88  
오하늘,84  
임수빈,90  
서도현,79  
배가은,95  
신준호,82  
조예린,87  
문태윤,74  
장하린,91  
고유진,86  
차민석,78  
류나영,89  
송시현,83  
김하윤,92  
이현우,77  
박지민,85  
최유나,94  
강민준,80  
백서진,88  
남지호,73  
노아린,90  
권도윤,84

# Проверка правильности результатов

1. Общее число студентов равно 25, и это верно (очевидно)
2. Проверка среднего балла, получено 84.76 (полное совпадение)

```
scores = [  
    81, 93, 76, 88, 84,  
    90, 79, 95, 82, 87,  
    74, 91, 86, 78, 89,  
    83, 92, 77, 85, 94,  
    80, 88, 73, 90, 84  
]  
average = sum(scores) / len(scores)  
print("Средний балл", average)
```

Средний балл 84.76

3. Студентов с баллом не ниже среднего 13 человек, каждое имя проверяется по отдельности (очевидно)

Этот документ подготовлен в формате **Quarto Markdown**(.qmd), который позволяет объединять код **Python** и текст документа в одном файле, и выведен в формате PDF.